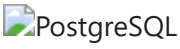
PostgreSQL Query Optimization



Introduction

Why should we care about optimizing our queries? Unlike imperative languages, where we state how we want to go about achieving a particular result, SQL is a declarative language. That is, when we write a query we are declaring the result we wish to achieve without much thought into telling the system how to go about producing that result. The system determines what it perceives the best path to take using a query optimizer that can be wrong if the data is irregular or the optimizer just doesn't have all of the neccessary information to choose the correct path in every situation. Most users when training to query data using SQL are only trained to find that result, because performance improvemnts can be minimal when working with small training datasets. However, when we enter real word scenarios dealing with Big Data, performance can quickly suffer as a result of writing inefficient queries. Someone who has felt themselves to be a query whiz could be placed into a position writing queries for a backend database or a data science application and quickly find themselves hamstringed by a system that seems to be moving slower than molases! As a result, badly written queries can cause several issues for an organization including:

- Resource bottlenecks across the system.
- Timout failures for users waiting for a response.
- Lost customer-base due to users who think a website or application is just too slow and clunky.
- Lost time, which can be catastrophic for applications performing real time data analysis.
- Developer bandwidth loss due to the time lost waiting for results to return, resulting in less time for research and ingenuity.

You may ask, what do we do to make our queries as performant as possible. Some may answer that we need to break a query down so far that we essentially make the SQL query imperative, where we are telling the system what result we want and we are defining exactly the execution plan that we want the system to follow. While this method may work in some cases, in many it can produce sub-optimal results. Like other SQL engines, we can lean on PostgreSQL's cost-based query optimizer that tries to turn your SQL queries into something efficient that executes in as little time as possible. Unlike other SQL engines, however, the PostgreSQL team has developed a philosophy of improving the built-in query optimizer as much as possible; even to the detriment of removing common functionality like 'hints' which allow you to specify exactly how you wish the query to be executed. Because of this, with each release, more and more optimization features are being added to the query optimizer; eliminating the need to control your queries at a micro level. In this practice notebook, we will cover how to use the planner to analyze your queries and restructure them in such a way that your success will be ensured. Let's get started!

Preparing Your Practice Environment

Loading Your Environment

To begin our explanation of PostgreSQL optimization techniques, we setup a database called airline_demo with some sample data that we can work with:

In [2]:
```
import pandas as pd
```

```
import os
from IPython.display import Markdown as md

pd.set_option('max_colwidth', 250)

# Please enter the information from the vocareum UI by pressing 'Start Lab' and clicking t
password = ''
host = ''

connection_string = "postgresql://postgres:{}@{}/airline_demo".format(password, host)

print("Connecting to PostgreSQL Database ({})...".format(connection_string))

%load_ext sql
%sql $connection_string

print("Finished connection process!")
```

```
Connecting to PostgreSQL Database (postgresql://postgres:divinity@localhost/airline_dem
o)...
Finished connection process!
```

We have also created a helper function that will format some results for this practice notebook in a more readable way.

In [3]:

```
from subprocess import PIPE, run

def run_explain_query(query, connection_url):
    result = ''
    query = 'psql -c "{}" {}'.format(query.replace('\n',''), connection_url)
    result = run(query, stdout=PIPE, stderr=PIPE, universal_newlines=True, shell=True)
    print(result.stdout)
```

Environment Overview

As you may have guessed, the loaded data is mock data for an imaginary airline containing about a month's worth of flight booking data. We can see the tables which have been built by querying the `pg_catalog.pg_tables` table:

In [4]:

```
%%sql

SELECT tablename
FROM pg_catalog.pg_tables
WHERE schemaname != 'pg_catalog' AND
    schemaname != 'information_schema';
```

```
 * postgresql://postgres:***@localhost/airline_demo
8 rows affected.
```

Out[4]:

tablename
ticket_flights
boarding_passes
aircrafts_data
flights
airports_data
seats
tickets

bookings

By running a quick query against the pg_catalog.pg_tables table, we can see that 7 tables have been setup for our use. The tables, follow the schema below:



Regarding the data, the main entity is a booking (`bookings`).

One booking can include several passengers, with a separate ticket (`tickets`) issued to each passenger. A ticket has a unique number and includes information about the passenger. As such, the passenger is not a separate entity. Both the passenger's name and identity document number can change over time, so it is impossible to uniquely identify all the tickets of a particular person; for simplicity, we can assume that all passengers are unique.

The ticket includes one or more flight segments (`ticket_flights`). Several flight segments can be included into a single ticket if there are no non-stop flights between the points of departure and destination (`connecting flights`), or if it is a round-trip ticket. Although there is no constraint in the schema, it is assumed that all tickets in the booking have the same flight segments.

Each flight (`flights`) goes from one airport (`airports`) to another. Flights with the same flight number have the same points of departure and destination, but differ in departure date.

At flight check-in, the passenger is issued a boarding pass (`boarding_passes`), where the seat number is specified. The passenger can check in for the flight only if this flight is included into the ticket. The flight-seat combination must be unique to avoid issuing two boarding passes for the same seat.

The number of seats (`seats`) in the aircraft and their distribution between different travel classes depends on the model of the aircraft (`aircrafts`) performing the flight. It is assumed that every aircraft model has only one cabin configuration. Database schema does not check that seat numbers in boarding passes have the corresponding seats in the aircraft (such verification can be done using table triggers, or at the application level).

We can get a quick look at the size of our data with a few quick queries:

In [3]:
```
result = %sql SELECT \
        (SELECT COUNT(*) FROM ticket_flights) AS ticket_flights, \
        (SELECT COUNT(*) FROM boarding_passes) AS boarding_passes, \
        (SELECT COUNT(*) FROM aircrafts_data) AS aircrafts_data, \
        (SELECT COUNT(*) FROM flights) AS flights, \
        (SELECT COUNT(*) FROM airports_data) AS airports_data, \
        (SELECT COUNT(*) FROM seats) AS seats, \
        (SELECT COUNT(*) FROM tickets) AS tickets, \
        (SELECT COUNT(*) FROM bookings) AS bookings;
```

```
 * postgresql://postgres:***@localhost/airline_demo
1 rows affected.
```

In [4]:
```
df = result.DataFrame().set_index(pd.Index(['count']))
df["Total"] = df.sum(axis = 1)[0]
df
```

Out[4]:

	ticket_flights	boarding_passes	aircrafts_data	flights	airports_data	seats	tickets	bookings	Total
count	1045726	579686	9	33121	104	1339	366733	262788	2289506

We have nearly 2.3 Million rows of data that we can utilize for our exploration into performance techniques! Before we get started optimizing queries, however, let's take a quick look at explain plans and how we can use them as a tool for helping us optimize our queries.

Execution Plans and the Explain Statement

What is an Execution Plan?

As mentioned in our introduction. The PostgreSQL engine uses a query optimizer to put together a plan for producing a result based on a user-provided query by applying transformation rules, heuristics, and cost-based optimization algorithms. The optimizer can do a lot of the heavy lifting when you are working to build an efficient query, allowing you more time to concentrate on data and less time waiting for results. The plan that the optimizer creates from analyzing your query is called the execution plan.

How to see the Execution Plan?

Execution plans can be view by using an Explain command paired with any grammatically correct SQL statement. This query can be ran directly in the PostgreSQL query editor. For this notebook, however, we will utilize the `run_explain_query(query, connection_string)` helper function to get our output into a more readable state. For the helper function, `query` is the SQL query to be ran and `connection_string` is the our PostgreSQL connection string. For example:

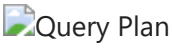
In [7]:
```
query = """
EXPLAIN
    SELECT
        a.book_ref,
        b.passenger_id,
        b.passenger_name
    FROM
        bookings a
    LEFT JOIN
        tickets b
    ON
        a.book_ref = b.book_ref;"""

run_explain_query(query, connection_string)
```

```
                                  QUERY PLAN
------------------------------------------------------------------------------
 Hash Right Join  (cost=8613.73..26146.75 rows=366733 width=35)
   Hash Cond: (b.book_ref = a.book_ref)
   ->  Seq Scan on tickets b  (cost=0.00..9811.33 rows=366733 width=35)
   ->  Hash  (cost=4301.88..4301.88 rows=262788 width=7)
         ->  Seq Scan on bookings a  (cost=0.00..4301.88 rows=262788 width=7)
(5 rows)
```

Reading an Execution Plan

The execution plan is printed as a tree, where, often, each leaf of the tree is denoted by the `->` . Executing via SQL magic seems to drop the arrow from the root process. The code executed above can give us the following output:


Query Plan

The output is a tree where each node of the tree is a operation performed during the query, found on separate lines starting with a `->`. The depth of the node is represented by the offset of the line from the left. Some operations, such as the Hash Right Join are represented across two lines, listing things like hash conditions. The plan executes starting from the leaves of the tree and ends at the root, meaning the line with the rightmost offset. Interpretting our plan above, the sequence of events is as follows:

1. A `bookings` table is scanned for all of the information that we need.
2. The `tickets` table is scanned for all of the information that we need.
3. That information from the `bookings` table and `tickets` table is loaded into a hash table.
4. The information compiled into the Hash table is joined together based on our hash condition.

Now that we understand the order of events, you may have notices the numbers to the right of the operations. As the PostgreSQL engine doesn't exactly run the query when putting together a plan, these are predicted statistic for running each operation. Take the scan on tickets, for example:

```
Seq Scan on tickets b  (cost=0.00..9811.33 rows=366733 width=35)
```

The first section, `cost=0.00..9811.33`, provides information regarding the cumulative cost running of the operation. In PostgreSQL, th optimizer uses internal metrics based on the amount of computing resources needed to execute a query or a single physical operation within a plan. The most important resources are those that affect execution time, namely, CPU cycles and I/O accesses (read/write disk blocks). Other resources, such as memory or disk space, have an indirect impact on execution time; for example, the amount of available memory will influence the ratio of CPU cycles and I/O accesses. The distribution of memory is controlled by server parameters and will not be covered in this text. In this case, we are provided with two values, 0.00 and 9811.33. The first value is an estimate of cost needed to produce the first row of output, while the second value is the estimated cost to produce the complete result. The next section is `rows=366733`, which gives us the expected number of rows in the output. Then, `width=35` gives us the estimated average width of those rows.

As we mentioned, the queries aren't actually ran with explain statements and these statistic are only estimates generated from information the engine has stored regarding the data in the tables, so how do we find out what the actual cost is for running each operation. In this case, we can use the `ANALYZE` statement with our explain query:

In [136…

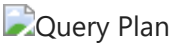
```
query = """
EXPLAIN ANALYZE
    SELECT
        a.book_ref,
        b.passenger_id,
        b.passenger_name
    FROM
        bookings a
    LEFT JOIN
        tickets b
    ON
        a.book_ref = b.book_ref;"""

run_explain_query(query, connection_string)
```

```
                                                                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------
 Hash Right Join  (cost=8613.73..26146.75 rows=366733 width=35) (actual time=31.011..207.017 rows=366733 loops=1)
   Hash Cond: (b.book_ref = a.book_ref)
```

```
   ->  Seq Scan on tickets b  (cost=0.00..9811.33 rows=366733 width=35) (actual time=0.03
2..31.670 rows=366733 loops=1)
   ->  Hash  (cost=4301.88..4301.88 rows=262788 width=7) (actual time=30.597..30.598 rows=
262788 loops=1)
         Buckets: 131072  Batches: 4  Memory Usage: 3519kB
         ->  Seq Scan on bookings a  (cost=0.00..4301.88 rows=262788 width=7) (actual time
=0.008..11.584 rows=262788 loops=1)
 Planning Time: 0.988 ms
 Execution Time: 212.983 ms
(8 rows)
```

When we run an `Explain Analyze` command, the query is actually ran and statistics regarding its execution are gathered:



Let's take another look at the scan on bookings:

```
  Seq Scan on tickets b  (cost=0.00..9811.33 rows=366733 width=35) (actual
time=0.032..31.670 rows=366733 loops=1)
```

We now have a second set of data: `(actual time=0.032..31.670 rows=366733 loops=1)` , giving the actual startup and completion time in milliseconds, actual numbers of rows returned, and the number of times the operation looped.

Why Change Anything?

After explaining execution plans and all of the work the optimizer does to make our queries as performant as possible, you may be asking yourself why we should change anything with a query to get it to run faster. The truth is that there isn't a single possible execution plan for a query, in fact, there are often hundreds, thousands, or even millions depending on the query! While PostgreSQL chooses the best plan it can find based on total cost, it can't check every single plan for every query. Heuristics are used to cut out parts of the plan universe that have less possibility of containing the best plan. However, there is still a possibility that the best plan is getting dropped! How do we ensure that the algorithms choose the best plan? We provide the query in the best form that allows for the optimizer to find the most cost-efficient plan.

Next, database systems maintain statistics for data contained in a table in order to calculate cost. If these statistic become stale and aren't updated as data is loaded, the optimizer may be calculating the wrong cost. we can remedy this by calculating statistics for our tables using the `ANALYZE` command:

```
ANALYZE table;
```

Now, try running your own explain statement.

YOU TRY

Follow the prompts to practice with `EXPLAIN` commands.

1. Run an `EXPLAIN ANALYZE` command on the provided query.

In [18]:

```
query = """
SELECT
        passenger_id,
        passenger_name
```

```
    FROM
        tickets
    GROUP BY
        passenger_id,
        passenger_name;"""
```

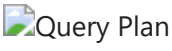
In [20]:
```
# 1


# Solution
run_explain_query("EXPLAIN ANALYZE {}".format(query), connection_string)
```

```
                                                                 QUERY PLAN

-------------------------------------------------------------------------------------------------------------
-----------------------------
 HashAggregate  (cost=39035.37..47716.62 rows=366733 width=28) (actual time=105.936..179.6
17 rows=366733 loops=1)
   Group Key: passenger_id, passenger_name
   Planned Partitions: 16  Batches: 17  Memory Usage: 4241kB  Disk Usage: 30744kB
   ->  Seq Scan on tickets  (cost=0.00..9811.33 rows=366733 width=28) (actual time=0.034..
20.996 rows=366733 loops=1)
 Planning Time: 0.586 ms
 Execution Time: 215.715 ms
(6 rows)
```

Given the produced execution plan below, answer the remaining questions.



2. What is the first operation performed?

In [ ]:
```
# 2

# Solution
# Seq Scan
```

3. What was the total estimated cost run finish running the query?

In [ ]:
```
# 3

# Solution

# 47716.62
```

4. What was the total execution time of the query?

In [21]:
```
# 4

# Solution

# 179.617 ms
```

5. How many operations were performed?

In [25]:
```
# 5

# Solution
```

```
# 2
```

6. Which operation performed the grouping?

In [26]:
```
# 6

# Solution

# HashAggregate
```

PostgreSQL Indexes

What are Indexes

When you need to find information in a book do you tend to read the whole book to find what you are looking for, or do you check the table of contents or index to find the quickest way to get to the information you need? Most people tend to check the table of contents or index! Indexes in PostgreSQL operate much like the index of a book, in that they are special lookup tables that a database can use to speed up data retrieval when filtering table queries on indexed columns. As such, a properly indexed table can vastly improve the performance of many queries. The vast majority of the time, these indexes are created by database engineers for many reasons such as:

- Understanding all of the technicalities and the different types of indexes that can be created could often be considered a specialized skillset.
- Index creation statements often take much longer to run, as opposed to typical queries.
- Indexes must be updated after every data load, therefore, it will often be most beneficial to include index creation and update as part of the ETL (Extract-Transform-Load) data ingestion process.

While database engineers are usually responsible for creating and maintaining indexes, there may come times when you don't have an index for column of a table you may need for a non-efficient query, or maybe you are working with data in a large table containing a subset of data from a production table that doesn't have indexes managed by data engineers. In times such as these, a high-level understanding of indexes and their creation is very beneficial. In this section, we will:

- Provide a brief high-level summary of Index types and their benefits.
- Show how to look up indexes by database and by table.
- Explain and demonstrate the creation of the default B-tree index type
- Show the performance benefit of an index.
- Explain how to drop an index.

Afterwards, you will get the chance to create and drop your own Index.

Types of Indexes

PostgreSQL has six primary index types that are available for utilization. Each type uses a different algorithm and is for different types of queries:

- B-tree (default) - B-tree is a self-balancing tree that maintains sorted data and allows searches, insertions, deletions, and sequential access in logarithmic time. PostgreSQL will consider using this index whenever

indexed columns are involved in comparisons using the following operators (`<` , `<=` , `=` , `>=` , `BETWEEN` , `IN` , `IS NULL` , `IS NOT NULL`) and queries using `LIKE` or `~` .

- Hash - Hashed indexes can only handle simple equality comparisons using `=` .
- Generelized Inverted Indexes (GIN) - GIN indexes are most useful when multiple values are stored in a single column, such as `hstore` , `array` , `jsonb` , and `range` type columns.
- Block Range Indexes (BRIN) - BRIN indexes are much small and less expensive to maintained when compared with B-tree. These indexes are used on very large tables where maintaining a B-tree index could prove impracticle. This index is also usually used for columns with linear sort order, such as columns containing date data.
- Generalized Search Tree GiST - GiST indexes are useful for geometric data types and full-text search.
- Space-Partitioned GiST (SP-GiST) - most useful for data that has a natural clustering element to it and is also not an equally balanced tree, such as GIS, multimedia, phone routing, and IP routing data.

For each of the types above, you have the option of:

- Single-Column Indexes - Indexes based on one column.
- Multicolumn Indexes - Indexes based on more than one columns. If most queries use more than one column of a table for filtering, then using a multicolumn index can be very beneficial.
- Unique Indexes - Indexes usually that not only increase query performance, but also can perserve data integrity by not allowing duplicate values.
- Partial Indexes - Indexes built over a subset of the table based on a conditional statement given in the Index creation statement.
- Implicit Indexes - Indexes automatically created by the database server when objects are created, such as unique indexes.

As we are keeping this discussion high-level, we will not be diving into the rabbit hole of how to create correct indexes for optimizing specific data types and query types. Let's get started by seeing what we already have.

Viewing Indexes

We can view any currently existing indexes by running a query on the `pg_catalog.pg_indexes` table and filtering that result based on table names in our database retrieved via a query to the `pg_catalog.pg_tables table` .

In [35]:

```
%%sql

SELECT
    tablename,
    indexname,
    indexdef
FROM
    pg_catalog.pg_indexes
WHERE
    tablename
IN
    (SELECT
        tablename
     FROM
        pg_catalog.pg_tables
     WHERE
        schemaname != 'pg_catalog'
     AND
        schemaname != 'information_schema')
```

```
 * postgresql://postgres:***@localhost/airline_demo
11 rows affected.
```

Out[35]:

tablename	indexname	indexdef
ticket_flights	ticket_flights_pkey	CREATE UNIQUE INDEX ticket_flights_pkey ON bookings.ticket_flights USING btree (ticket_no, flight_id)
boarding_passes	boarding_passes_flight_id_boarding_no_key	CREATE UNIQUE INDEX boarding_passes_flight_id_boarding_no_key ON bookings.boarding_passes USING btree (flight_id, boarding_no)
boarding_passes	boarding_passes_flight_id_seat_no_key	CREATE UNIQUE INDEX boarding_passes_flight_id_seat_no_key ON bookings.boarding_passes USING btree (flight_id, seat_no)
boarding_passes	boarding_passes_pkey	CREATE UNIQUE INDEX boarding_passes_pkey ON bookings.boarding_passes USING btree (ticket_no, flight_id)
aircrafts_data	aircrafts_pkey	CREATE UNIQUE INDEX aircrafts_pkey ON bookings.aircrafts_data USING btree (aircraft_code)
flights	flights_flight_no_scheduled_departure_key	CREATE UNIQUE INDEX flights_flight_no_scheduled_departure_key ON bookings.flights USING btree (flight_no, scheduled_departure)
flights	flights_pkey	CREATE UNIQUE INDEX flights_pkey ON bookings.flights USING btree (flight_id)
airports_data	airports_data_pkey	CREATE UNIQUE INDEX airports_data_pkey ON bookings.airports_data USING btree (airport_code)
seats	seats_pkey	CREATE UNIQUE INDEX seats_pkey ON bookings.seats USING btree (aircraft_code, seat_no)
tickets	tickets_pkey	CREATE UNIQUE INDEX tickets_pkey ON bookings.tickets USING btree (ticket_no)
bookings	bookings_pkey	CREATE UNIQUE INDEX bookings_pkey ON bookings.bookings USING btree (book_ref)

At the moment, we have 11 indexes currently existing. You will notice from their index definition column, `indexdef` that they are unque indexes from the `CREATE UNIQUE INDEX` syntax. Unique indexes are created during table creation for increased performance when filtering on primary keys and to preserve data integrity by not allowing duplicate values. Let's try to create our own index.

Index Creation

Let's use the `flights` table for our exploration into indexes. Take for example the following query:

In [50]:

```
%%timeit -r 1 -n 1

%%sql

select arrival_airport, departure_airport, status from flights where arrival_airport = 'CN
```

```
 * postgresql://postgres:***@localhost/airline_demo
3 rows affected.
3.21 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

An index is created using the following syntax:

```
CREATE INDEX index_name on table_name (column(s)_to_index);
```

Let's create a multi-column index for `arrival_airport` and `departure_airport` .

In [51]:
```
%%timeit -r 1 -n 1

%%sql

CREATE INDEX flights_arr_dep_index on flights (arrival_airport, departure_airport);
```

```
 * postgresql://postgres:***@localhost/airline_demo
Done.
80.4 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

Now, let's rerun our original query to check for any performance gain:

In [52]:
```
%%timeit -r 1 -n 1

%%sql

select arrival_airport, departure_airport, status from flights where arrival_airport = 'CN
```

```
 * postgresql://postgres:***@localhost/airline_demo
3 rows affected.
2.27 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

At the time of writing this practice exercise, we noticed around a 30% improvement in processing time. That can be huge when you are dealing with queries that take hours to run!

If we choose to drop this index and test creating other indexes until we find one we like the most, we can do so with the following syntax:

```
DROP INDEX index_name;
```

Let's drop the index we just created:

In [53]:
```
%%sql

DROP INDEX flights_arr_dep_index;
```

```
 * postgresql://postgres:***@localhost/airline_demo
Done.
```

Out[53]:
```
[]
```

Now, you can try creating your own index.

YOU TRY

Follow the prompts to create a drop your own index:

1. Create an index named `flight_status_index` for the column `status` in the flights table.

In [71]:
```
%%sql

/*solution code*/
CREATE INDEX flight_status_index ON flights (status);
```

```
 * postgresql://postgres:***@localhost/airline_demo
Done.
```

Out[71]:
```
[]
```

2. Drop the `flight_status_index` index.

In [72]:
```
%%sql

/*solution code*/
DROP INDEX flight_status_index;
```

```
 * postgresql://postgres:***@localhost/airline_demo
Done.
```

Out[72]: []

Optimizing PostgreSQL Queries

Long and Short Queries

The strategy for optimizing PostgreSQL starts with identifying if a query is 'long' or 'short'. You may be wondering what that means... It has nothing to do with how long the query actually is! What actually identifies whether a query is long or short is the number of rows needed to compute its output.

> A query is labeled as short when the number of rows needed to compute the output is small. They may read every row from a lot of small tables, but only a small percentage of rows from large tables. As a rule of thumb, short queries only operate on less than 10% of the rows from big tables.

> A query is labeled as long when the query selectivity is high for at least one of the large tables. Even if the output size is relatively small, if the query has to perform full scans across entire large table datasets resulting in many more rows being involved in the computation of the output, it is labeled as long.

Take the following two queries for example:

In [29]:
```
%%sql

select fare_conditions, count(fare_conditions) from ticket_flights group by fare_condition
```

```
 * postgresql://postgres:***@localhost/airline_demo
3 rows affected.
```

Out[29]:

fare_conditions	count
Business	107642
Comfort	17291
Economy	920793

In [31]:
```
%%sql

select * from flights limit 5;
```

```
 * postgresql://postgres:***@localhost/airline_demo
5 rows affected.
```

Out[31]:

flight_id	flight_no	scheduled_departure	scheduled_arrival	departure_airport	arrival_airport	status	aircraft_cod
1185	PG0134	2017-09-10 01:50:00-05:00	2017-09-10 06:55:00-05:00	DME	BTK	Scheduled	31
3979	PG0052	2017-08-25 06:50:00-	2017-08-25	VKO	HMA	Scheduled	CR

		05:00	09:35:00-05:00				
4739	PG0561	2017-09-05 04:30:00-05:00	2017-09-05 06:15:00-05:00	VKO	AER	Scheduled	76
5502	PG0529	2017-09-12 01:50:00-05:00	2017-09-12 03:20:00-05:00	SVO	UFA	Scheduled	76
6938	PG0461	2017-09-04 04:25:00-05:00	2017-09-04 05:20:00-05:00	SVO	ULV	Scheduled	SU

In [37]:

```
query = """Explain \
    SELECT \
        a.ticket_no, \
        a.flight_id, \
        b.scheduled_arrival, \
        b.scheduled_departure \
    FROM \
        ticket_flights a \
    JOIN \
        flights b \
    ON \
        a.flight_id = b.flight_id;"""

run_explain_query(query, connection_string)
```

```
                                QUERY PLAN
--------------------------------------------------------------------------------
 Hash Join  (cost=1137.22..23054.87 rows=1045726 width=34)
   Hash Cond: (a.flight_id = b.flight_id)
   ->  Seq Scan on ticket_flights a  (cost=0.00..19172.26 rows=1045726 width=18)
   ->  Hash  (cost=723.21..723.21 rows=33121 width=20)
         ->  Seq Scan on flights b  (cost=0.00..723.21 rows=33121 width=20)
(5 rows)
```

In [36]:

```
query = """Explain \
    SELECT \
        a.ticket_no, \
        a.flight_id, \
        b.scheduled_arrival, \
        b.scheduled_departure \
    FROM \
        (SELECT \
            ticket_no, \
            flight_id \
         FROM \
            ticket_flights \
         WHERE \
            fare_conditions = 'Comfort') a \
    JOIN \
        flights b \
    ON \
        a.flight_id = b.flight_id;"""

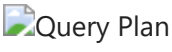
run_explain_query(query, connection_string)
```

```
                                QUERY PLAN
--------------------------------------------------------------------------------
-
 Gather  (cost=2137.22..17905.60 rows=15895 width=34)
   Workers Planned: 2
```

```
   ->  Hash Join  (cost=1137.22..15316.10 rows=6623 width=34)
         Hash Cond: (ticket_flights.flight_id = b.flight_id)
         ->  Parallel Seq Scan on ticket_flights  (cost=0.00..14161.49 rows=6623 width=18)
               Filter: ((fare_conditions)::text = 'Comfort'::text)
         ->  Hash  (cost=723.21..723.21 rows=33121 width=20)
               ->  Seq Scan on flights b  (cost=0.00..723.21 rows=33121 width=20)
(8 rows)
```

With both of our query plans in place, we can check to identify our query type. Let's first take a look at our first query.

```
SELECT
   a.ticket_no,
   a.flight_id,
   b.scheduled_arrival,
   b.scheduled_departure
FROM
   ticket_flights a
JOIN
   flights b
ON
   a.flight_id = b.flight_id;
```


Query Plan

While this query is more simple than our second query, it runs a scan across the entire `ticket_flights` table:

```
Seq Scan on ticket_flights a  (cost=0.00..19172.26 rows=1045726 width=18)
```

For this reason, we say that this query is a long query. Consider our second query:

```
SELECT
   a.ticket_no,
   a.flight_id,
   b.scheduled_arrival,
   b.scheduled_departure
FROM
   (SELECT
      ticket_no,
      flight_id
   FROM
      ticket_flights
   WHERE
      fare_conditions = 'Comfort') a
JOIN
   flights b
ON
   a.flight_id = b.flight_id;
```



The filter we applied for `fare_conditions` in our selection statement for the `ticket_flights` table resulted in us only selecting a fraction of the records that the `ticket_flights` . Because we are only pulling a small fraction of the data from `ticket_flights` and the overall size of `flights` is relatively small compared to our other big tables, we can say that this is a short query.

Next, try identifying if a query is long or short on your own.

YOU TRY

Given the following query, is this query a long or short query?

```
SELECT
   count(b.seat_no)
FROM
   aircrafts_data a
JOIN
   seats b
ON
   a.aircraft_code = b.aircraft_code;
```

In [48]:
```
# Solution

"""While this query scans whole tables, it is only working with the smaller tables of our
```

Out[48]:
```
'While this query scans whole tables, it is only working with the smaller tables of our da
tabase. For this reason, it is actually a short query.'
```

Optimizing Short Queries

Short Query Optimization Goals

When working with short queries, we know that we are dealing with a relatively small number of records. Because of the lower number of input records into our query, we don't really focus on join optimization as much as we would with long queries. Instead, our optimization goal becomes retrieveing those records as quickly as possible by using our indexes. The best performance for short queries often occurs when we use the most restrictive indexes with the lowest selectivity in order to speed up the search. We start are optimization by finding our most selective selection criteria in our query.

Finding the Most Restrictive Criteria

Our first step to optimizing a short query is to find our most restrictive selection criteria. Consider the following query:

```
SELECT
   flight_id,
   departure_airport,
   arrival_airport,
```

```
  scheduled_departure,
  scheduled_arrival,
  status
FROM
  flights
WHERE
  arrival_airport = 'VKT'
AND
  scheduled_departure
BETWEEN
  '2017-08-01' and '2017-09-01';
```

For this query, we are providing two different selection criteria:

- `arrival_airport = 'VKT'`
- `scheduled_departure BETWEEN '2017-08-01' and '2017-09-01'`

We can check for the most selective criteria but running a few quick counts on the flights table:

In [13]:
```
%%sql

SELECT count(*) FROM flights;
```

```
 * postgresql://postgres:***@localhost/airline_demo
1 rows affected.
```

Out[13]:

count
33121

In [10]:
```
%%sql

SELECT count(*) FROM flights WHERE arrival_airport = 'VKT';
```

```
 * postgresql://postgres:***@localhost/airline_demo
1 rows affected.
```

Out[10]:

count
279

In [12]:
```
%%sql

SELECT count(*) FROM flights WHERE scheduled_departure BETWEEN '2017-08-01' and '2017-09-0
```

```
 * postgresql://postgres:***@localhost/airline_demo
1 rows affected.
```

Out[12]:

count
16825

As you can see. The arrival_airport filter cuts our row size down from 33,121 records to only 279, while the scheduled_departure filter only cuts the row size down to 16,825. In this case, we would say that the arrival_airport filter provides our most restrictive selection criteria. Great! So, what now?

Next, we would need to check our indexes in order to be sure that there was an index in place for this criteria.

Checking the Indexes

Once we have our most restrictive selection criteria, we want to check if we have an index for those columns in this table. We can do so with a quick query to the `pg_catalog` table:

In [14]:
```
%%sql

SELECT
    tablename,
    indexname,
    indexdef
FROM
    pg_catalog.pg_indexes
WHERE
    tablename
IN
    (SELECT
        tablename
     FROM
        pg_catalog.pg_tables
     WHERE
        schemaname != 'pg_catalog'
     AND
        schemaname != 'information_schema'
     AND
        tablename = 'flights')
```

```
 * postgresql://postgres:***@localhost/airline_demo
2 rows affected.
```

Out[14]:

tablename	indexname	indexdef
flights	flights_flight_no_scheduled_departure_key	CREATE UNIQUE INDEX flights_flight_no_scheduled_departure_key ON bookings.flights USING btree (flight_no, scheduled_departure)
flights	flights_pkey	CREATE UNIQUE INDEX flights_pkey ON bookings.flights USING btree (flight_id)

It looks as if we have two indexes for our `flights` table, neither of them on `arrival_airport`. At this point, you would want to create an index for that column or reach out to a data engineer to design the best index for you query specifically. In order to get an idea of the benefit, let's take a look at execution time before index creation:

In [17]:
```
%%timeit -r 1 -n 1
%%sql

SELECT
    flight_id,
    departure_airport,
    arrival_airport,
    scheduled_departure,
    scheduled_arrival,
    status
FROM
    flights
WHERE
    arrival_airport = 'VKT'
AND
    scheduled_departure
BETWEEN
    '2017-08-01' and '2017-09-01';
```

```
 * postgresql://postgres:***@localhost/airline_demo
```

```
142 rows affected.
4.26 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

As of running this code on our system, we received an execution time of 4.26 ms

- `4.26 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)`

Now, let's create a new index and get a look at the new execution time!

In [18]:
```
%%sql

CREATE INDEX arrival_airport_index ON flights (arrival_airport);
```

```
 * postgresql://postgres:***@localhost/airline_demo
Done.
```

Out[18]:
```
[]
```

In [19]:
```
%%timeit -r 1 -n 1
%%sql

SELECT
    flight_id,
    departure_airport,
    arrival_airport,
    scheduled_departure,
    scheduled_arrival,
    status
FROM
    flights
WHERE
    arrival_airport = 'VKT'
AND
    scheduled_departure
BETWEEN
    '2017-08-01' and '2017-09-01';
```

```
 * postgresql://postgres:***@localhost/airline_demo
142 rows affected.
2.34 ms ± 0 ns per loop (mean ± std. dev. of 1 run, 1 loop each)
```

Now, we are receiving an execution time of 2.34 ms! That is nearly 50%. While 2 ms may not seem like a lot, when a query is ran millions of times a day it adds up quickly! We can also check our explain plan to ensure that our index is being used:

In [22]:
```
query = """Explain \
SELECT \
    flight_no, \
    departure_airport, \
    arrival_airport, \
    scheduled_departure, \
    scheduled_arrival, \
    status \
FROM \
    flights \
WHERE \
    arrival_airport = 'VKT' \
AND \
    scheduled_departure \
BETWEEN \
    '2017-08-01' and '2017-09-01';"""

run_explain_query(query, connection_string)
```

```
                                                
```

Any time we use any kind of transformation function on a column, we technically change the data and it will no longer be indexed. For this section, we will consider 3 examples of functions that will cause us to bypass using an index and how to alter a query to adjust for that. First, let's consider date data. Take the following query, for example, where we query flight data based on scheduled departure:

```
SELECT
   flight_id,
   scheduled_departure
FROM
   flights
WHERE
   scheduled_departure ::date
BETWEEN
   '2017-08-01' and '2017-08-05';
```

Let's create a quick index on `scheduled_departure` and run our query:

In [24]:
```
%%sql

CREATE INDEX scheduled_departure_index ON flights(scheduled_departure);
```

```
 * postgresql://postgres:***@localhost/airline_demo
Done.
[]
```

Out[24]:

In [34]:
```
query = """Explain Analyze \
SELECT \
    flight_id, \
    scheduled_departure \
FROM \
    flights \
WHERE \
    scheduled_departure ::date \
BETWEEN \
    '2017-08-01' and '2017-08-05';"""

run_explain_query(query, connection_string)
```

```
                                                      QUERY PLAN

------------------------------------------------------------------------------------------------------
------------------------------
 Seq Scan on flights  (cost=0.00..1054.42 rows=166 width=12) (actual time=0.014..4.031 row
s=2702 loops=1)
   Filter: (((scheduled_departure)::date >= '2017-08-01'::date) AND ((scheduled_departur
e)::date <= '2017-08-05'::date))
   Rows Removed by Filter: 30419
 Planning Time: 1.005 ms
 Execution Time: 4.092 ms
(5 rows)
```

From our explain plan, running the query performs a `Seq Scan` on the flights table, instead of an Index scan, returning 2702 rows of data in 4.092 ms. The reason for this is due to us casting `scheduled_departure` as a date type. Instead of doing so, we can leave it as is, and let PostgreSQL do a string comparison:

In [35]:
```
query = """Explain Analyze \
SELECT \
    flight_id, \
    scheduled_departure \
FROM \
    flights \
WHERE \
    scheduled_departure \
BETWEEN \
    '2017-08-01' and '2017-08-05';"""

run_explain_query(query, connection_string)
```

```
                                                                                                  QUERY
PLAN
-----------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on flights  (cost=38.91..464.02 rows=2207 width=12) (actual time=0.150..
1.052 rows=2136 loops=1)
   Recheck Cond: ((scheduled_departure >= '2017-08-01 00:00:00-05'::timestamp with time zo
ne) AND (scheduled_departure <= '2017-08-05 00:00:00-05'::timestamp with time zone))
   Heap Blocks: exact=385
   ->  Bitmap Index Scan on scheduled_departure_index  (cost=0.00..38.36 rows=2207 width=
0) (actual time=0.117..0.118 rows=2136 loops=1)
         Index Cond: ((scheduled_departure >= '2017-08-01 00:00:00-05'::timestamp with tim
e zone) AND (scheduled_departure <= '2017-08-05 00:00:00-05'::timestamp with time zone))
 Planning Time: 0.776 ms
 Execution Time: 1.111 ms
(7 rows)
```

Rerunning the adjusted query shows us that the index scan is now being performed, returning the same number of rows in 1.111 ms. That means our adjusted query has ran in nearly a quarter of the time!

Next, let's consider the `coalesce()` function. Running this function on a query will also result in the index not being used.

```
SELECT
  *
FROM
  flight
WHERE
  coalesce(actual_departure, scheduled_departure)
BETWEEN
  '2020-08-17' AND '2020-08-18';
```

Let's create an index on `actual_departure` and run an explain to get a look at this:

In [36]:
```
%%sql

CREATE INDEX actual_departure_index ON flights(actual_departure);
```

```
 * postgresql://postgres:***@localhost/airline_demo
Done.
[]
```

Out[36]:

In [37]:
```
query = """Explain Analyze \
```

```
SELECT \
    * \
FROM \
    flights \
WHERE \
    coalesce(actual_departure, scheduled_departure) \
BETWEEN \
    '2017-08-01' and '2017-08-05';"""

run_explain_query(query, connection_string)
```

```
                  QUERY PLAN
-----------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------
-------------------------------------------
 Seq Scan on flights  (cost=0.00..888.82 rows=166 width=63) (actual time=0.028..2.740 rows
=2138 loops=1)
   Filter: ((COALESCE(actual_departure, scheduled_departure) >= '2017-08-01 00:00:00-05'::
timestamp with time zone) AND (COALESCE(actual_departure, scheduled_departure) <= '2017-08
-05 00:00:00-05'::timestamp with time zone))
   Rows Removed by Filter: 30983
 Planning Time: 1.021 ms
 Execution Time: 2.792 ms
(5 rows)
```

From our explain query, we can see that we are not accessing our index, and we are receiving 2138 rows of data with an execution time of 2.792 ms. If we adust our query just a bit, we can avoid the use of coalesce and make our query much more performant:

In [38]:

```
query = """Explain Analyze \
SELECT \
    * \
FROM \
    flights \
WHERE \
    (actual_departure BETWEEN '2017-08-01' and '2017-08-05') \
OR \
    (actual_departure IS NULL AND scheduled_departure BETWEEN '2017-08-01' and '2017-08-05

run_explain_query(query, connection_string)
```

```
                                                                                QUERY P
LAN
-----------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------
 Bitmap Heap Scan on flights  (cost=78.04..557.76 rows=3194 width=63) (actual time=0.240..
1.127 rows=2138 loops=1)
   Recheck Cond: (((actual_departure >= '2017-08-01 00:00:00-05'::timestamp with time zon
e) AND (actual_departure <= '2017-08-05 00:00:00-05'::timestamp with time zone)) OR ((sche
duled_departure >= '2017-08-01 00:00:00-05'::timestamp with time zone) AND (scheduled_depa
rture <= '2017-08-05 00:00:00-05'::timestamp with time zone)))
   Filter: (((actual_departure >= '2017-08-01 00:00:00-05'::timestamp with time zone) AND
(actual_departure <= '2017-08-05 00:00:00-05'::timestamp with time zone)) OR ((actual_depa
rture IS NULL) AND (scheduled_departure >= '2017-08-01 00:00:00-05'::timestamp with time z
one) AND (scheduled_departure <= '2017-08-05 00:00:00-05'::timestamp with time zone)))
   Heap Blocks: exact=385
```

```
   ->  BitmapOr  (cost=78.04..78.04 rows=4386 width=0) (actual time=0.211..0.211 rows=0 lo
ops=1)
         ->  Bitmap Index Scan on actual_departure_index  (cost=0.00..38.08 rows=2179 widt
h=0) (actual time=0.149..0.149 rows=2138 loops=1)
               Index Cond: ((actual_departure >= '2017-08-01 00:00:00-05'::timestamp with
time zone) AND (actual_departure <= '2017-08-05 00:00:00-05'::timestamp with time zone))
         ->  Bitmap Index Scan on scheduled_departure_index  (cost=0.00..38.36 rows=2207 w
idth=0) (actual time=0.061..0.061 rows=2136 loops=1)
               Index Cond: ((scheduled_departure >= '2017-08-01 00:00:00-05'::timestamp wi
th time zone) AND (scheduled_departure <= '2017-08-05 00:00:00-05'::timestamp with time zo
ne))
 Planning Time: 1.102 ms
 Execution Time: 1.229 ms
(11 rows)
```

We are now accessing our data with index scans, retrieving the same number of rows, and doing so in less than half the time!

You may not always want to rewrite your query in a strange way to work around using functions. There is a solution to that as well! You can create an index based on the transformed value. Consider the case of filtering data based on last name. A common transformation applied here is to convert the name to lower case in order to ensure the data matches. Consider this query:

```
SELECT
  *
FROM
  tickets
WHERE
  LOWER(passenger_name)
LIKE
  'niko%';
```

In [46]:
```
query = """Explain Analyze SELECT * FROM tickets WHERE lower(passenger_name) LIKE 'niko%';

run_explain_query(query, connection_string)
```

```
                                                          QUERY PLAN

-------------------------------------------------------------------------------------------
---------------------------------
 Gather  (cost=1000.00..9619.48 rows=1834 width=104) (actual time=0.224..79.021 rows=8048
loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Parallel Seq Scan on tickets  (cost=0.00..8436.08 rows=764 width=104) (actual time=
0.129..65.851 rows=2683 loops=3)
         Filter: (lower(passenger_name) ~~ 'niko%'::text)
         Rows Removed by Filter: 119562
 Planning Time: 0.483 ms
 Execution Time: 79.190 ms
(8 rows)
```

Running the code above, we can see that a `Seq Scan` is used as opposed to an index scan, with 8048 rows returned in 79.190 ms. We can create an index that works for our purposes here, by creating an index based on the transformed values of `passenger_name`:

In [144…

```
%%sql

CREATE INDEX lower_name_index ON tickets(lower(passenger_name) text_pattern_ops);
DROP INDEX actual_departure_index;
DROP INDEX scheduled_departure_index;
```

```
 * postgresql://postgres:***@localhost/airline_demo
Done.
Done.
Done.
Done.
```

Out[144…

```
[]
```

In [49]:

```
query = """Explain Analyze SELECT * FROM tickets WHERE lower(passenger_name) LIKE 'niko%';

run_explain_query(query, connection_string)
```

```
                                                                                                    QUERY PLAN

-------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on tickets  (cost=31.22..4002.42 rows=1834 width=104) (actual time=0.986..17.495 rows=8048 loops=1)
   Filter: (lower(passenger_name) ~~ 'niko%'::text)
   Heap Blocks: exact=4484
   ->  Bitmap Index Scan on lower_name_index  (cost=0.00..30.76 rows=1834 width=0) (actual time=0.628..0.628 rows=8048 loops=1)
         Index Cond: ((lower(passenger_name) ~>=~ 'niko'::text) AND (lower(passenger_name) ~<~ 'nikp'::text))
 Planning Time: 0.688 ms
 Execution Time: 17.684 ms
(7 rows)
```

Running the query again, we can see that we still get 8048 rows return, but we are getting our result using an index scan in 17.648 ms! That's about 22% of the original time!

Optimizing Long Queries

Short Query Optimization Goals

Now that we have covered optimizing short queries, we can now explore optimizing long queries. When working with long queries, we are working with A LOT of input data because we are performing either full or near-full scans of our large tables and often performing joins. Because of the size of the input data, utilizing indexes to speed up our queries becomes inefficient due to the cost of I/O operations associated with crossreferencing an index table over a large number of rows. Because of this, our focus for optimization, instead becomes optimizing our joins. Instead of first finding the most restrictive selection criteria, we find our most restrictive join to begin with in order to reduce the size of our dataset as soon as we can in our query process.

Finding the Most Restrictive Join

Before beginning, semi-joins and anti-joins are the most performant joins in PostgreSQL when it comes to reducing the size of the resulting data set. The two joins can be defined as such:

> A semi-join, like a left outer join, is a join between two tables R and S, returning rows from table R for which there is at least one row from table s with matching values in the joining columns.

A anti-join, is the opposite of the semi-join. It is a join between two tables R and S, returning rows from table R for which there are no rows from table S with a matching value in the joining column.

A semi-join query can be written as follows:

```
SELECT * FROM flights f WHERE EXISTS
   (SELECT flight_id FROM ticket_flights WHERE flight_id=f.flight_id)
```

As expected, an anti-join query is writtent with almost the same syntax, except with a `NOT` modifier:

```
SELECT * FROM flights f WHERE NOT EXISTS
   (SELECT flight_id FROM ticket_flights WHERE flight_id=f.flight_id)
```

Some analysts may have the habit of using the IN and $NOTIN$ condition instead of $WHEREEXISTS$ or $WHERENOTEXISTS$, however, the optimizer will more often choose semi or anti-join using the $WHERE$ syntax rather than the IN syntax.

PostgreSQL's optimizer should choose the join that produces the smallest output to occur first automatically, however, you will want to check that with the query execution plan. For example, take the following query:

```
SELECT * FROM
   bookings
JOIN tickets USING (book_ref)
JOIN ticket_flights USING (ticket_no)
JOIN flights USING (flight_id);
```

In [56]:
```
%%sql

SELECT COUNT(*) FROM bookings JOIN tickets USING (book_ref);
```

```
 * postgresql://postgres:***@localhost/airline_demo
1 rows affected.
```

Out[56]:

count
366733

In [57]:
```
%%sql

SELECT COUNT(*) FROM tickets JOIN ticket_flights USING (ticket_no);
```

```
 * postgresql://postgres:***@localhost/airline_demo
1 rows affected.
```

Out[57]:

count
1045726

In [58]:
```
%%sql
```

```
SELECT COUNT(*) FROM ticket_flights JOIN flights USING (flight_id);
```

```
 * postgresql://postgres:***@localhost/airline_demo
1 rows affected.
```

Out[58]:

count
1045726

Based on the above, we can see that the most restrictive join is that between `bookings` and `tickets`. Now let's check to see if that join is processed first:

In [55]:

```
query = """Explain Analyze \
SELECT * FROM \
    bookings \
JOIN tickets USING (book_ref) \
JOIN ticket_flights USING (ticket_no) \
JOIN flights USING (flight_id);"""

run_explain_query(query, connection_string)
```

```
                                                                                                  QUERY PLAN
------------------------------------------------------------------------------------------------------------------------------------------------
 Hash Join  (cost=45071.13..102112.51 rows=1045726 width=195) (actual time=335.757..1041.419 rows=1045726 loops=1)
   Hash Cond: (ticket_flights.flight_id = flights.flight_id)
   ->  Hash Join  (cost=43933.91..98229.90 rows=1045726 width=136) (actual time=328.574..879.313 rows=1045726 loops=1)
         Hash Cond: (ticket_flights.ticket_no = tickets.ticket_no)
         ->  Seq Scan on ticket_flights  (cost=0.00..19172.26 rows=1045726 width=32) (actual time=0.010..66.055 rows=1045726 loops=1)
         ->  Hash  (cost=32902.75..32902.75 rows=366733 width=118) (actual time=327.869..327.871 rows=366733 loops=1)
               Buckets: 32768  Batches: 16  Memory Usage: 3672kB
               ->  Hash Join  (cost=9126.73..32902.75 rows=366733 width=118) (actual time=33.292..261.529 rows=366733 loops=1)
                     Hash Cond: (tickets.book_ref = bookings.book_ref)
                     ->  Seq Scan on tickets  (cost=0.00..9811.33 rows=366733 width=104) (actual time=0.006..31.406 rows=366733 loops=1)
                     ->  Hash  (cost=4301.88..4301.88 rows=262788 width=21) (actual time=32.740..32.741 rows=262788 loops=1)
                           Buckets: 65536  Batches: 8  Memory Usage: 2271kB
                           ->  Seq Scan on bookings  (cost=0.00..4301.88 rows=262788 width=21) (actual time=0.005..9.227 rows=262788 loops=1)
   ->  Hash  (cost=723.21..723.21 rows=33121 width=63) (actual time=7.061..7.061 rows=33121 loops=1)
         Buckets: 65536  Batches: 1  Memory Usage: 3617kB
         ->  Seq Scan on flights  (cost=0.00..723.21 rows=33121 width=63) (actual time=0.011..3.178 rows=33121 loops=1)
 Planning Time: 2.863 ms
 Execution Time: 1058.391 ms
(18 rows)
```

If we were to rearrange the join, we can see that the optimizer still chooses the correct order for the joins:

In [59]:

```
query = """Explain Analyze \
SELECT * FROM \
    flights \
JOIN ticket_flights USING (flight_id) \
JOIN tickets USING (ticket_no) \
```

```
JOIN bookings USING (book_ref);"""

run_explain_query(query, connection_string)
```

```
                                                                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------------------
 Hash Join  (cost=45071.13..102112.51 rows=1045726 width=195) (actual time=333.248..999.845 rows=1045726 loops=1)
   Hash Cond: (ticket_flights.flight_id = flights.flight_id)
   ->  Hash Join  (cost=43933.91..98229.90 rows=1045726 width=136) (actual time=328.874..848.642 rows=1045726 loops=1)
         Hash Cond: (ticket_flights.ticket_no = tickets.ticket_no)
         ->  Seq Scan on ticket_flights  (cost=0.00..19172.26 rows=1045726 width=32) (actual time=0.009..64.393 rows=1045726 loops=1)
         ->  Hash  (cost=32902.75..32902.75 rows=366733 width=118) (actual time=328.073..328.075 rows=366733 loops=1)
               Buckets: 32768  Batches: 16  Memory Usage: 3723kB
               ->  Hash Join  (cost=9126.73..32902.75 rows=366733 width=118) (actual time=39.635..260.184 rows=366733 loops=1)
                     Hash Cond: (tickets.book_ref = bookings.book_ref)
                     ->  Seq Scan on tickets  (cost=0.00..9811.33 rows=366733 width=104) (actual time=0.005..31.027 rows=366733 loops=1)
                     ->  Hash  (cost=4301.88..4301.88 rows=262788 width=21) (actual time=39.105..39.106 rows=262788 loops=1)
                           Buckets: 65536  Batches: 8  Memory Usage: 2239kB
                           ->  Seq Scan on bookings  (cost=0.00..4301.88 rows=262788 width=21) (actual time=0.006..14.264 rows=262788 loops=1)
   ->  Hash  (cost=723.21..723.21 rows=33121 width=63) (actual time=4.206..4.206 rows=33121 loops=1)
         Buckets: 65536  Batches: 1  Memory Usage: 3716kB
         ->  Seq Scan on flights  (cost=0.00..723.21 rows=33121 width=63) (actual time=0.009..1.524 rows=33121 loops=1)
 Planning Time: 2.839 ms
 Execution Time: 1016.831 ms
(18 rows)
```

What do we do if the join order the optimizier chooses is wrong without hints, however? The optimizer will only develop an execution plan for a maximum number of tables, set by the variable `join_collapse_limit` . If a join involves any more tables than that limit, the joins are performed in the order they are given. Therefore, we can override the optmizer by setting the limit to 1 before running our query using:

- `SET join_collapse_limit=1;`

Basically, this will disable the optimizer, allowing you to manually specify the order. This method should rarely be used, however, and well formed queries should execute properly. After, optimizing your joins, the next step is to check your grouping and filters.

Grouping and Filtering

Grouping is an expensive operation that can quickly raise the cost of any given query. For that reason, except in times where grouping would bastly reduce an intermediate dataset between joins, you will usually want to ensure that all grouping is occuring after all of your filters have been applied in order to limit the amount of I/O that occurs. Therefore, you will want to push filters inside the grouping. For example, consider the following query written with grouping occuring first:

```
SELECT * FROM
   (SELECT arrival_airport, COUNT(arrival_airport) FROM flights GROUP BY arrival_airport) a
WHERE
   arrival_airport = 'VKT';
```

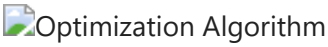
In [68]:
```
query = "EXPLAIN ANALYZE SELECT * FROM (SELECT arrival_airport, COUNT(arrival_airport) FRO
run_explain_query(query, connection_string)
```

```
                                                                                                  QUERY PLAN

------------------------------------------------------------------------------------------------------------------------
------------------------------------------------------
 GroupAggregate  (cost=0.29..11.63 rows=97 width=12) (actual time=0.157..0.157 rows=1 loop
s=1)
   Group Key: flights.arrival_airport
   ->  Index Only Scan using arrival_airport_index on flights  (cost=0.29..9.24 rows=283 w
idth=4) (actual time=0.131..0.140 rows=279 loops=1)
         Index Cond: (arrival_airport = 'VKT'::bpchar)
         Heap Fetches: 0
 Planning Time: 1.329 ms
 Execution Time: 0.202 ms
(7 rows)
```

The optimizer will rewrite the query to actually perform the grouping last, as seen in th explain plan above. If grouping isn't occurring last and the query has lag issues, more work may need to be done to investigate the filters.

Optimization Algorithm



Practice on Your Own

Now that you've learned about optimizing queries with PostgreSQL. Try optimizing one on your own by following the prompts!

Consider the following query:

```
SELECT
   a.book_date,
   a.total_amount,
   b.passenger_id,
   b.passenger_name,
   c.ticket_no,
   c.fare_conditions,
   d.scheduled_arrival,
   d.scheduled_departure
FROM
   bookings a
```

```
JOIN tickets b USING(book_ref)
JOIN ticket_flights c USING(ticket_no)
JOIN flights d USING(flight_id)
WHERE
   d.status = 'Scheduled'
AND
   d.scheduled_departure
BETWEEN
   '2017-08-15' AND '2017-08-18';
```

1. Print the execution plan for this query

In [154…

```
#1


# Solution

query = """Explain Analyze \
SELECT \
    a.passenger_id, \
    a.passenger_name, \
    b.ticket_no, \
    b.fare_conditions, \
    c.scheduled_arrival, \
    c.scheduled_departure \
FROM \
    tickets a \
JOIN ticket_flights b USING(ticket_no) \
JOIN flights c USING(flight_id) \
WHERE \
    c.status = 'Scheduled' \
AND
    b.fare_conditions = 'Business' \
AND \
    c.scheduled_departure \
BETWEEN \
    '2017-08-15' AND '2017-08-18';"""
run_explain_query(query, connection_string)
```

```
                     QUERY PLAN

-----------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------
-----------------------------------------------------
 Gather  (cost=1981.50..17243.68 rows=2455 width=66) (actual time=2.929..41.596 rows=2667
loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Nested Loop  (cost=981.50..15998.18 rows=1023 width=66) (actual time=2.560..27.155
rows=889 loops=3)
         ->  Hash Join  (cost=981.08..15260.04 rows=1023 width=38) (actual time=2.532..22.
937 rows=889 loops=3)
               Hash Cond: (b.flight_id = c.flight_id)
               ->  Parallel Seq Scan on ticket_flights b  (cost=0.00..14161.49 rows=44748
width=26) (actual time=0.011..19.149 rows=35881 loops=3)
                     Filter: ((fare_conditions)::text = 'Business'::text)
                     Rows Removed by Filter: 312695
               ->  Hash  (cost=971.62..971.62 rows=757 width=20) (actual time=2.475..2.475
rows=633 loops=3)
```

```
                      Buckets: 1024  Batches: 1  Memory Usage: 41kB
                      ->  Seq Scan on flights c  (cost=0.00..971.62 rows=757 width=20) (act
ual time=0.097..2.409 rows=633 loops=3)
                            Filter: ((scheduled_departure >= '2017-08-15 00:00:00-05'::time
stamp with time zone) AND (scheduled_departure <= '2017-08-18 00:00:00-05'::timestamp with
time zone) AND ((status)::text = 'Scheduled'::text))
                            Rows Removed by Filter: 32488
         ->  Index Scan using tickets_pkey on tickets a  (cost=0.42..0.72 rows=1 width=42)
(actual time=0.005..0.005 rows=1 loops=2667)
               Index Cond: (ticket_no = b.ticket_no)
 Planning Time: 2.858 ms
 Execution Time: 41.696 ms
(18 rows)
```

2. What is the execution time of this query?

In [136…

```
#2

# Solution

'Variable, but 41.142 ms in this case.'
```

Out[136…

```
'Variable, but 41.142 ms in this case.'
```

3. How many rows does this query return?

In [137…

```
#3

# Solution

2667
```

Out[137…

```
2667
```

4. Is this a short query or a long query?

In [81]:

```
#4

# Solution

'Short Query'
```

Out[81]:

```
'Short Query'
```

5. What is the most restrictive selection criteria for this query?

In [ ]:

```
#5

# Solution

"SELECT count(*) FROM flights where scheduled_departure BETWEEN '2017-08-15' AND '2017-08-
```

6. Create an index for the most restrictive selection criteria.

In [158…

```
#6

# Solution
```

```
%sql CREATE INDEX scheduled_departure_index ON flights(scheduled_arrival);
```

```
                        QUERY PLAN

------------------------------------------------------------------------------------------------
------------------------------------------------------------------------------------------------
-------------------------------------------------------
 Gather  (cost=1981.50..17243.68 rows=2455 width=66) (actual time=2.739..40.850 rows=2667
loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Nested Loop  (cost=981.50..15998.18 rows=1023 width=66) (actual time=2.529..27.015
rows=889 loops=3)
         ->  Hash Join  (cost=981.08..15260.04 rows=1023 width=38) (actual time=2.500..22.
846 rows=889 loops=3)
               Hash Cond: (b.flight_id = c.flight_id)
               ->  Parallel Seq Scan on ticket_flights b  (cost=0.00..14161.49 rows=44748
width=26) (actual time=0.015..19.114 rows=35881 loops=3)
                     Filter: ((fare_conditions)::text = 'Business'::text)
                     Rows Removed by Filter: 312695
               ->  Hash  (cost=971.62..971.62 rows=757 width=20) (actual time=2.437..2.438
rows=633 loops=3)
                     Buckets: 1024  Batches: 1  Memory Usage: 41kB
                     ->  Seq Scan on flights c  (cost=0.00..971.62 rows=757 width=20) (act
ual time=0.078..2.370 rows=633 loops=3)
                           Filter: ((scheduled_departure >= '2017-08-15 00:00:00-05'::time
stamp with time zone) AND (scheduled_departure <= '2017-08-18 00:00:00-05'::timestamp with
time zone) AND ((status)::text = 'Scheduled'::text))
                           Rows Removed by Filter: 32488
         ->  Index Scan using tickets_pkey on tickets a  (cost=0.42..0.72 rows=1 width=42)
(actual time=0.004..0.004 rows=1 loops=2667)
               Index Cond: (ticket_no = b.ticket_no)
 Planning Time: 2.184 ms
 Execution Time: 40.948 ms
(18 rows)
```

7. What is the new execution time?

In [88]:
```
# 7

# Solution

'41.135 ms'
```

Out[88]:
```
'1.192 ms'
```

8. Create a partial filter for when `fare_conditions='Business'` on the `ticket_flights` table.

In [159…
```
# 8

# Solution

%sql CREATE INDEX business_fare_index ON ticket_flights(ticket_no) WHERE fare_conditions='
```

Out[159…
```
 * postgresql://postgres:***@localhost/airline_demo
Done.
[]
```

9. Print the new execution plan.

In [164…

```
# 9

# Solution

query = """Explain Analyze \
SELECT \
    a.passenger_id, \
    a.passenger_name, \
    b.ticket_no, \
    b.fare_conditions, \
    c.scheduled_arrival, \
    c.scheduled_departure \
FROM \
    tickets a \
JOIN ticket_flights b USING(ticket_no) \
JOIN flights c USING(flight_id) \
WHERE \
    c.status = 'Scheduled' \
AND
    b.fare_conditions = 'Business' \
AND \
    c.scheduled_departure \
BETWEEN \
    '2017-08-15' AND '2017-08-18';"""
run_explain_query(query, connection_string);
```

```
QUERY PLAN

------------------------------------------------------------------------------------------------------------------------------------------------------------
 Gather  (cost=3818.99..14194.03 rows=2455 width=66) (actual time=18.537..30.718 rows=2667 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Nested Loop  (cost=2818.99..12948.53 rows=1023 width=66) (actual time=7.990..16.764 rows=889 loops=3)
         ->  Hash Join  (cost=2818.57..12210.40 rows=1023 width=38) (actual time=7.956..12.449 rows=889 loops=3)
               Hash Cond: (b.flight_id = c.flight_id)
               ->  Parallel Bitmap Heap Scan on ticket_flights b  (cost=2132.25..11406.60 rows=44748 width=26) (actual time=1.529..8.853 rows=35881 loops=3)
                     Recheck Cond: ((fare_conditions)::text = 'Business'::text)
                     Heap Blocks: exact=4947
                     ->  Bitmap Index Scan on business_fare_index  (cost=0.00..2105.40 rows=107396 width=0) (actual time=3.900..3.900 rows=107642 loops=1)
               ->  Hash  (cost=676.86..676.86 rows=757 width=20) (actual time=1.954..1.955 rows=633 loops=3)
                     Buckets: 1024  Batches: 1  Memory Usage: 41kB
                     ->  Index Scan using scheduled_status_index on flights c  (cost=0.29..676.86 rows=757 width=20) (actual time=0.017..1.888 rows=633 loops=3)
                           Filter: ((scheduled_departure >= '2017-08-15 00:00:00-05'::timestamp with time zone) AND (scheduled_departure <= '2017-08-18 00:00:00-05'::timestamp with time zone))
                           Rows Removed by Filter: 14750
         ->  Index Scan using tickets_pkey on tickets a  (cost=0.42..0.72 rows=1 width=42) (actual time=0.005..0.005 rows=1 loops=2667)
               Index Cond: (ticket_no = b.ticket_no)
 Planning Time: 3.337 ms
 Execution Time: 30.809 ms
(19 rows)
```

10. What is the new execution time?

In [ ]:

```
#10

# Solution
'30.809 ms'
```